Malware Analysis: NSIS Dropper

What This File

This is a "dropper." It doesn't do the damage itself. It's a delivery box. It hides other files inside itself, drops them into your temp folder, and runs them. Attackers like using NSIS (a legit installer tool) to build these because it slips past basic antivirus checks more easily than a custom packer would.

1. Structure

File type: 32-bit Windows EXE (PE32), built with the GUI subsystem. Droppers like this usually run silently though, without showing a window.

Built with: NSIS (Nullsoft Scriptable Install System). You can tell from strings baked into the file like `NSIS Error` and `verifying installer: %d%`, plus the NSIS support URL.

Sections in the file:

- `.text`: the actual code
- `.rdata`: read only strings and data
- `.data`: variables the program writes to
- `.ndata`: uninitialized data (this one's a dead giveaway for NSIS)
- `.rsrc`: resources; this is where the hidden payload lives

Layout: The front of the file is just the NSIS installer engine. The real payload, an executable called `_Nb.exe` and a `.dll`, sits compressed inside the `.rsrc` section (or the overlay), out of sight from a quick scan.

2. FUNCTIONALITY

unpack the hidden files, drop them on disk, run them, maybe force a reboot.

A. File Operations

It finds your temp folder with `GetTempPathW`, then writes `_Nb.exe` and a `.dll` into it using `CreateFileW` and `WriteFile`. It also calls `SetFileAttributesW`, likely to hide the dropped files from a normal folder view.

`DeleteFileW` deletes a file. The dropper may use this to delete itself or clean up temporary extraction scripts once the payload is launched.

`FindFirstFileW`, `FindNextFileW`, and `FindClose` search a folder for files. The dropper uses these to confirm the files it dropped actually exist, or to clean up old files in the directory.

B. Process Manipulation (Execution and Monitoring)

Once the payload is on disk, the dropper wakes it up.

`CreateProcessW` is the core function used to start a new program. The malware passes the path to `_Nb.exe` to this function, telling Windows to run it.

`ShellExecuteExW` is an alternative way to launch programs. It uses the `open` verb, found in the strings. This tells Windows to open `_Nb.exe` exactly as if a user had double clicked it.

`WaitForSingleObject` tells the dropper to pause and wait. It waits until the newly launched `_Nb.exe` has finished its initial execution before continuing its own cleanup.

`GetExitCodeProcess` checks how the child process exited once it finishes, whether cleanly or with a crash. The dropper uses this to decide what to do next.

`GetCurrentProcess` returns a handle to the dropper itself. Used for self manipulation, such as adjusting its own privileges.

`ExitProcess` is used by the dropper to cleanly terminate itself once its job of dropping and launching the payload is complete.

C. System and Privilege Manipulation (Forcing Reboots)

The malware wants to ensure it has the power to restart your computer if necessary.

`OpenProcessToken` opens a "token" that represents the security context of the running dropper. Think of it as checking the dropper's security badge.

`LookupPrivilegeValueW` looks up the specific security ID for a privilege. The strings show it is looking for `SeShutdownPrivilege`, the right to shut down the computer.

`AdjustTokenPrivileges` takes that `SeShutdownPrivilege` ID and turns it on for the dropper. Now it has the power to reboot the PC.

`ExitWindowsEx` and `InitiateShutdownW` actually trigger the shutdown or reboot. Malware usually does this to force newly installed drivers or persistence mechanisms, like registry Run keys, to activate.

D. Registry Modifications

The malware interacts with the Windows Registry to check settings or install persistence.

`RegOpenKeyExW` opens a specific registry folder, called a key. The strings show it opens `Software\Microsoft\Windows\CurrentVersion`.

`RegQueryValueExW` reads a specific value inside that registry key. Likely done to check system configuration or see if the machine is already infected.

`RegSetValueExW` writes a new value to the registry. This is a classic persistence trick. By writing the path to `_Nb.exe` into the Run key, the malware makes sure it starts automatically every time the computer turns on.

`RegCreateKeyExW` creates a brand new registry key if one doesn't already exist.

`RegDeleteValueW` and `RegDeleteKeyW` delete registry entries, likely to clean up traces or remove competing malware.

`RegEnumKeyW` and `RegEnumValueW` loop through registry entries to find specific data.

E. Environment Awareness and Evasion

The dropper checks its surroundings to make sure it isn't being analyzed in a sandbox.

`GetVersionExW` asks Windows for the exact OS version. Malware checks this to avoid running on older Windows versions where it might crash, or to dodge security features in newer versions.

`GetUserDefaultUILanguage` checks the language settings of the computer. Malware often uses this to avoid infecting machines in certain countries, to stay out of reach of local law enforcement.

`GetDiskFreeSpaceW` checks how much free space is on the hard drive. Sandboxes and virtual machines often have unusually small drives, so if the disk looks too small, the malware may stop running to avoid analysis.

`LoadLibraryExW`, `GetModuleHandleW`, and `GetProcAddress` load Windows DLLs into memory dynamically. Loading APIs this way instead of statically hides the malware's true capabilities from basic static analysis tools.

`GetSystemDirectoryW` and `GetWindowsDirectoryW` find the `C:\Windows\System32` and `C:\Windows` folders, used to know where legitimate Windows files are stored.

Execution Order

1. NSIS engine starts and checks it hasn't been tampered with. If it has, it throws `NSIS Error` and stops.
2. It calls `GetVersionExW` and `GetUserDefaultUILanguage` to check the OS and language, and `GetDiskFreeSpaceW` to check disk space.
3. It finds the temp directory with `GetTempPathW` and creates a staging file with `GetTempFileNameW`.
4. It decompresses its payload and uses `CreateFileW` and `WriteFile` to drop `_Nb.exe` and a `.dll` into the temp folder, possibly hiding them with `SetFileAttributesW`.
5. It calls `OpenProcessToken` and `AdjustTokenPrivileges` to enable `SeShutdownPrivilege`, preparing for a reboot.
6. It runs `_Nb.exe` using `CreateProcessW` or `ShellExecuteExW` with the `open` argument.
7. It waits for `_Nb.exe` to finish its initial setup using `WaitForSingleObject`.
8. It likely uses `DeleteFileW` to remove its temporary files, then calls `ExitWindowsEx` to reboot the system and finalize the install.